POSITION PAPER · ON-DEVICE LEARNING · PRIVACY

Behavioral Observation as a Training Signal

A Mechanism for Privacy-Preserving Personal Adapter Calibration Without Supervision, Labels, or Transmitted Data

Jonathan Horvat

2026 · PUBLIC DOMAIN · NO RIGHTS RESERVED

> *This paper is a companion to:* Horvat, J. (2026). *Intelligence at the Edge of the Cloud: A Local Architecture for the AI Era.* Public domain.

ABSTRACT

The edge AI architecture proposed in the companion paper claims that personal adapter weights train through behavioral observation — without supervised labels, explicit feedback, or transmitted data. This paper closes that specification gap. We define the Behavioral Preference Signal (BPS): a set of implicit interaction events — query reformulation, session continuation depth, dwell time, and follow-up query structure — that together constitute a noisy but aggregate-reliable proxy for user preference. We propose a constrained low-rank gradient update rule derived from Direct Preference Optimization that uses BPS scores as implicit preference labels, apply drift prevention constraints to prevent over-fitting and adversarial manipulation, and define a three-tier validation design. The mechanism is trainable on-device, requires no server communication, produces no supervisory signal that could be intercepted or compelled, and is grounded in established preference learning theory.

SECTION I

The Specification Gap

The companion architecture paper makes a precise claim: personal adapter weights update through behavioral observation only — no supervised loss, no explicit feedback, no transmitted gradient. The user is never asked to rate anything. The adapters become calibrated through the natural shape of interaction.

That claim is architecturally motivated and privacy-correct. But it is underspecified.

What exactly does the system observe? What constitutes a training signal? How does an observation translate into a weight update? What prevents the adapter from drifting toward a bad state if the observations are noisy or adversarially crafted?

These are not rhetorical questions. They are engineering requirements. An architecture that leaves them unanswered is a proposal, not a blueprint. This paper answers them.

The key insight is that behavioral observation is not a new idea — it is the implicit form of the preference learning that explicit methods like RLHF and DPO already do. The difference is who provides the preference signal. In RLHF, a human annotator compares two responses and clicks a button. In the mechanism proposed here, the user's natural interaction behavior provides the same information — not explicitly, not consistently, but reliably in aggregate. The system observes what the user does after receiving a response, and treats that behavior as implicit evidence about whether the response served the user's need.

> *The preference signal has always been in the interaction. Most systems just never looked for it.*

→ [1, 2, 3]

SECTION II

The Behavioral Preference Signal

The Behavioral Preference Signal (BPS) is a scalar score assigned to each query-response pair based on the user's next observable action. It is computed on-device, immediately after the action occurs, and is never transmitted. It serves as the implicit preference label that drives the adapter update.

The signal is drawn from four observable event types, each with a time window and a score:

BEHAVIORAL PREFERENCE SIGNAL — EVENT TYPES AND SCORES

COMPUTED ON-DEVICE · NEVER TRANSMITTED

EVENT	WINDOW	BPS SCORE	RATIONALE
Query reformulation — user rephrases the same intent	< 45 seconds of response	−1.0	Unambiguous negative signal. The response did not serve the user's need. The reformulation reveals what the response missed — level, framing, or domain.
Deep continuation — follow-up query builds on response without restating	< 90 seconds of response	+1.0	Strong positive signal. The user understood the response and extended it. The structure of the follow-up reveals what the response communicated successfully.
Session end after	After	+0.4	Weak positive. Extended engagement without

sustained engagement (> 3 exchanges)	final response		reformulation suggests the response pattern was broadly useful. Noisy — user may have simply run out of time.
Immediate session termination after response	< 15 seconds of response	−0.3	Weak negative. Could indicate the response was complete and satisfying, or that the user gave up. Treated as weak negative only when combined with short session history.
No action — user reads response, session pauses	> 120 seconds inactivity	0.0	Neutral. Insufficient signal to determine preference. No adapter update triggered.

The BPS is intentionally simple. More sophisticated signals — dwell time, scroll behavior, copy actions — could be incorporated but introduce complexity without proportional benefit. The two primary signals (reformulation and deep continuation) are sufficient to distinguish responses that served the user from those that did not, in aggregate, over many interactions.

Individual BPS observations are noisy. A user might reformulate because they changed their mind, not because the response failed. A user might continue a session despite a mediocre response because the task required it. The mechanism does not need individual observations to be accurate. It needs the aggregate signal over many interactions to be directionally correct — which empirical work on revealed preference theory suggests it will be, given sufficient interaction volume.

→ [2, 4, 5]

SECTION III

Grounding in Established Theory

The BPS mechanism is not invented from whole cloth. It is a privacy-preserving instantiation of two well-established learning frameworks: Direct Preference Optimization and revealed preference theory from economics.

Direct Preference Optimization

DPO (Rafailov et al., 2023) showed that RLHF's complex reward modeling pipeline can be replaced by a simple binary cross-entropy objective over preference pairs. Given two responses to the same query — one preferred, one dispreferred — DPO directly optimizes the policy to increase the relative probability of the preferred response. No reward model. No RL loop. Just a classification loss over pairs.

The BPS mechanism adapts DPO to the implicit, on-device setting. Instead of human annotators providing preference labels, the system derives implicit preference labels from behavioral events. A query-response pair that triggers deep continuation is the "preferred" response in the DPO sense. A pair that triggers reformulation is the "dispreferred" response. The update rule is a DPO-derived gradient step on the adapter

weights — not the full model, only the thin low-rank delta — in the direction that increases the relative probability of continuation-triggering responses over reformulation-triggering ones.

The critical difference from standard DPO is the source of the preference signal and the scope of the update. Standard DPO updates the full model on explicit human labels aggregated at a server. The BPS mechanism updates only adapter weights on implicit behavioral labels observed entirely on-device. The mathematical structure is the same. The privacy properties are categorically different.

Revealed Preference Theory

In economics, revealed preference theory holds that a person's true preferences can be inferred from their choices, even when those choices are not explicitly framed as preference statements. A consumer who repeatedly buys product A over product B reveals a preference for A, regardless of what they say when asked directly.

The BPS is an application of this principle to conversational AI interaction. A user who consistently continues sessions after responses of a certain type reveals a preference for that type, regardless of whether they would articulate it explicitly. The behavioral signal reveals the preference. The adapter learns from what the user does, not what the user says they want.

This grounding matters for two reasons. First, it provides a theoretical basis for why the aggregate signal is reliable even when individual observations are noisy — revealed preferences aggregate in the same direction as true preferences over sufficient data. Second, it connects the mechanism to a 70-year-old literature in economics and decision theory that has validated the approach in many domains.

→ [1, 3, 5, 6]

SECTION IV

The Update Rule

The update rule specifies how BPS scores translate into adapter weight changes. It has three components: the gradient direction, the magnitude constraint, and the recency weighting.

BPS ADAPTER UPDATE RULE — CONSTRAINED LOW-RANK GRADIENT STEP · ON-DEVICE ONLY

$$\Delta\theta = \eta \cdot \mathrm{clip}(\nabla_\theta L_{BPS}(\theta), \delta_{max}) \cdot w_t$$

θ — Personal adapter weights (low-rank matrices A and B in LoRA parameterization).

	The frozen base model is never updated. Only the adapter delta is modified.
`η (learning rate)`	Fixed small learning rate. Recommended: 1×10^{-5}. Small enough that individual noisy observations cannot cause significant drift. Larger than typical fine-tuning rates to allow meaningful calibration over weeks of interaction.
$L_{BPS}(\theta)$	BPS-weighted DPO loss. For each scored event pair (q, r_+, r_-) — where r_+ is the continuation-triggering response and r_- is the reformulation-triggering response — the loss increases the log-ratio of $p(r_+\|q)$ to $p(r_-\|q)$ under the adapter-modified policy, weighted by the absolute BPS score magnitude.
$\text{clip}(\cdot, \delta_{max})$	Gradient clipping with maximum norm $\delta_{max} = 0.01 \times \|\theta\|$. Prevents any single update from moving adapter weights more than 1% of their current norm. This is the primary safeguard against adversarial manipulation and catastrophic forgetting.
w_t (recency weight)	Exponential decay weight: $w_t = \exp(-\lambda(t_{now} - t_{event}))$ with $\lambda = 0.1$/day. Recent observations contribute more to each update. Events older than 30 days contribute less than 5% of their original weight. Prevents over-indexing on old interaction patterns as the user's needs evolve.

> **DPO loss:** The BPS-weighted DPO loss increases the log-ratio of the preferred response probability to the dispreferred response probability under the adapter-modified policy, regularized by a KL constraint (coefficient $\beta = 0.1$) that prevents the adapter from drifting too far from the frozen base model — equivalent to the standard DPO objective with BPS scores as implicit preference weights. **Pair construction:** BPS events are not always naturally paired. When a deep continuation occurs without a preceding reformulation, the system pairs the continuation response against the base model's response to the same query (computed offline). When a reformulation occurs, the reformulated query and the subsequent response are used to construct the positive pole. The base model response acts as the reference for the KL constraint in the DPO objective, preventing catastrophic drift from the general capability baseline.

PSEUDOCODE — ON-DEVICE BPS ADAPTER UPDATE LOOP (SIMPLIFIED FOR CLARITY)

```
for each interaction session:
    events = observe_session(query_response_pairs)
    scored = assign_bps(events)              # Table from Section II
    pairs  = construct_preference_pairs(scored)

    for (q, r_pos, r_neg, score) in pairs:
        if abs(score) < threshold:           # Skip near-zero signal
            continue
        grad = compute_bps_dpo_gradient(
            adapter_weights,
            q, r_pos, r_neg,
            base_model_ref,
            beta=0.1                         # KL constraint coefficient
        )
        clipped = clip_by_norm(grad, delta_max)
        w_t = recency_weight(event.timestamp)
        adapter_weights += eta * clipped * w_t

    # Drift check after each session
    if adapter_norm_delta() > 0.15 * initial_norm:
        eta *= 0.5                           # Halve learning rate
        log_warning("Drift threshold approached")
```

The pseudocode is intentionally implementable. Every variable is specified. The drift check is explicit. The base model reference is included. This is not a sketch — it is a complete algorithm that could be ported to a LoRA training loop on any device that supports gradient computation in the personal adapter layer.

→ [1, 7, 8]

SECTION V

Drift Prevention and Adversarial Robustness

The BPS mechanism has two vulnerability surfaces: natural drift from noisy signals, and adversarial manipulation by a user deliberately trying to steer the adapter in a particular direction.

Natural Drift

Noisy individual observations accumulate. A sequence of unusual sessions — travel, illness, a deadline — might produce a cluster of atypical behavioral signals that push adapter weights away from the user's normal pattern. The recency decay partially addresses this: old signals fade. But the cumulative drift constraint is the primary safeguard. If adapter weights have moved more than 15% of their initial norm from the base model, the learning rate halves automatically. This acts like a governor — the adapter continues learning but cannot drift far enough from the base model to lose general capability.

The 15% threshold is a design parameter, not a derived constant. A tighter threshold (5–10%) produces a more conservative adapter that calibrates slowly. A looser threshold (20–30%) allows more personalization but increases the risk of capability loss. The right value depends on deployment context and should be empirically validated in the human pilot study described in Section VI.

Adversarial Manipulation

A user who understands the mechanism could deliberately generate reformulation and continuation signals to steer the adapter. This is not a theoretical attack — it is a practical one that a technically sophisticated user could attempt.

The gradient clipping is the primary defense. A maximum norm constraint of 1% of adapter norm per update means that even a deliberate sequence of high-magnitude BPS events cannot move the adapter significantly in a single session. Sustained manipulation over many sessions would be needed to produce meaningful drift — and even then, the 15% cumulative norm constraint limits the achievable manipulation.

A sophisticated adversary with physical device access could in principle attempt sustained manipulation — but this is precisely the threat the hardware secure enclave is designed to address, reducing it from a remote software attack to a local physical one. The more important observation is that the attack surface is narrow. The adapter encodes the user's patterns, not external world knowledge. A user manipulating their own adapter produces an adapter that serves their manipulated preferences — which is arguably the correct behavior. The security concern is not that a user manipulates their own model, but that a malicious actor manipulates another user's model. That attack requires physical access to the device, which is the secure enclave's job to prevent.

→ [7, 8, 9]

SECTION VI

Validation Design

The BPS mechanism is a proposal, not a proof. Three validation tiers — synthetic, simulation, and human pilot — are required to establish that the mechanism works as described. This section specifies each tier in enough detail that a research team could implement it directly.

Tier 1 Synthetic Validation

No humans required

Setup: Generate a dataset of synthetic interaction sequences with known preference structure. Each sequence consists of 50–100 query-response pairs drawn from a preference-consistent distribution — some user type always prefers technical depth, another always prefers brevity, a third prefers analogical reasoning.

Procedure: Run the BPS mechanism on each sequence. Assign BPS scores based on simulated behavioral events generated from the known preference structure. Apply the update rule. After N interactions, measure whether adapter weights have moved in the direction that would increase performance on held-out preference-consistent queries from the same distribution.

Success criterion: Adapter performance on held-out queries improves by at least 5% relative to base model on preference-consistent tasks. Adapter does not degrade on general capability tasks. Drift remains within bounds.

```
Primary metric: Δ(preference-consistent accuracy) vs Δ(general accuracy) Secondary metric: Adapter norm delta after N interactions Dataset size: 5 synthetic user types × 100 interaction sequences × 50 pairs = 25,000 events
```

Tier 2 Simulation Validation

LLM as simulated user

Setup: Use an existing aligned LLM (GPT-4 class or equivalent) as a simulated user with a defined preference profile. The simulated user has a fixed "true preference" — for example, it prefers responses that use structured enumeration over prose. The preference is known to the evaluator but not encoded in the adapter at the start.

Procedure: Run interactions between the adapter-equipped model and the simulated user. The simulated user generates realistic reformulation or continuation signals based on whether each response matches its known preference profile. Apply BPS updates after each interaction. After 50, 100, and 200 interactions, evaluate whether the adapter model now matches the simulated user's preference profile better than the base model.

Success criterion: At 200 interactions, adapter model response style matches simulated user preference on blind evaluation. Reformulation rate decreases monotonically over interactions. PΔ score (from the IpMW companion paper) reaches at least +0.15.

```
Primary metric: PΔ at 50 / 100 / 200 interactions Secondary metric: Reformulation rate per 10-interaction window (should decrease) Simulated users: Minimum 3 distinct preference profiles
```

Tier 3 Human Pilot Study

5–10 participants · 2 weeks

Setup: Recruit 8–12 participants with diverse professional backgrounds and interaction styles. Each participant uses the system as their primary AI assistant for two weeks. The system is split — half of participants use the BPS-updated adapter (treatment group), half use a frozen adapter that receives no updates (control group). Neither group is told which condition they are in.

Procedure: At days 1, 7, and 14, administer a blind preference evaluation: present each participant with 20 matched query-response pairs — one generated by their adapter model, one by the base model — and ask which they prefer without revealing the source. Record reformulation rates throughout the study period. At study end, interview participants about their experience.

Success criterion: Treatment group participants prefer their adapter model responses over base model responses at statistically significant rates by day 14. Treatment group reformulation rate decreases over the study period. Control group shows neither effect.

```
Primary metric: Blind preference rate (treatment vs control) at day 14 Secondary metric: Reformulation rate slope over study period Sample size: 8–12 participants (power analysis suggests 8 is sufficient for the expected effect size) Ethics: IRB approval required · Participants informed they are evaluating AI personalization · No sensitive data retained Power instrumentation: Device power draw during adapter update cycles should be measured and reported — connecting to the IpMW companion paper metric.
```

Tier 1 can be completed with no external resources beyond compute. Tier 2 requires API access to a capable LLM. Tier 3 requires ethical approval and participant recruitment. The tiers are designed to be sequential — positive results at each tier justify the investment in the next. A research team with access to Tier 1 validation could publish that result alone as a meaningful contribution to the privacy-preserving personalization literature.

→ [2, 3, 10]

SECTION VII

Open Questions

This paper specifies the mechanism and validation design but does not close every question. The following open questions are honest gaps — not rhetorical — and should be treated as research directions rather than objections.

01 **Cold start — how many interactions before PΔ becomes meaningful?**

A freshly deployed adapter with empty weights has PΔ = 0. The mechanism requires interactions to accumulate before calibration begins. The simulation validation will reveal the interaction count required for meaningful PΔ improvement, but this is not yet known. 50 interactions? 200? The answer has significant implications for how useful the system is in the first days of deployment.

02 **Preference drift — what happens when the user's genuine preferences change?**

The recency decay is designed to allow the adapter to evolve as the user changes. But the optimal decay constant λ is not known. Too fast and the adapter oscillates; too slow and it fails to track genuine preference evolution. The human pilot study will provide data, but the right λ may vary by user and deployment context.

03 Cross-domain generalization — does adapter calibration in one domain transfer to another?

If a user primarily interacts in their professional domain, the adapter calibrates to their professional communication style. Does that calibration transfer when they ask questions in an unfamiliar domain? The LoRA parameterization suggests some generalization should occur, but the extent is unknown and depends on the overlap between domain representations in the base model.

04 Optimal drift threshold — is 15% the right cumulative norm constraint?

The 15% norm constraint is a design choice, not a derived constant. The right threshold balances personalization depth against capability preservation. Too tight and the adapter cannot meaningfully differentiate from the base model. Too loose and capability degradation becomes a real risk. Empirical validation across diverse user types is required to establish this threshold with confidence.

None of these open questions undermine the mechanism. They are the natural boundary of what a position paper can establish without empirical data. The validation design in Section VI is designed precisely to answer questions 01 through 04. The mechanism is specified with enough precision to run that validation today.

→ [2, 3, 10]

REFERENCES

[1] Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D. & Finn, C. (2023). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model.* NeurIPS 2023. arXiv:2305.18290. — Foundational DPO algorithm; the BPS update rule is a privacy-preserving on-device adaptation of this objective.

[2] Zhao, A. et al. (2025). *PrefEval: Do LLMs Recognize Your Preferences? Evaluating Personalized Preference Following in LLMs.* ICLR 2025 (Oral). — Demonstrates that current LLMs fail to follow implicit preferences at conversation scale; motivates the adapter-based approach.

[3] Li, J. et al. (2024). *Personalized Language Modeling from Personalized Human Feedback (P-RLHF).* ICLR 2025. — Closest existing work to BPS mechanism; handles implicit preferences but requires server-side aggregation and gradient transmission. BPS eliminates both requirements.

[4] Samuelson, P. A. (1948). *Consumption Theory in Terms of Revealed Preference.* Economica, 15(60), 243–253. — Foundational revealed preference theory; grounds the BPS behavioral signal in 70-year-old economic decision theory.

[5] Zollo, F. et al. (2024). *PersonalLLM: Tailoring LLMs to Individual Preferences.* ICLR 2025. — Benchmark for LLM personalization; establishes that diverse user preferences are learnable from interaction history.

[6] Hu, E. et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models.* arXiv:2106.09685. — Low-rank parameterization used for the adapter weights updated by the BPS mechanism.

[7] Kopiczko, D. J. et al. (2025). *Low-rank adaptation for edge AI.* Scientific Reports. — Edge-specific LoRA deployment; grounds the on-device training loop in production-validated methodology.

[8] Apple Machine Learning Research (2024/2025). *Apple Intelligence Foundation Language Models.* machinelearning.apple.com. — Production precedent for frozen base + LoRA adapter pattern on consumer hardware; validates the hardware feasibility of on-device adapter updates.

[9] Horvat, J. (2026). *Applying HAZOP to AI Systems.* Public domain. — HAZOP register for the full architecture identifies adapter over-training (Node 3, More guide word) as a Medium severity risk; the drift prevention

mechanism in Section V is the action required for that register entry.

[10] Tummalapalli, P. et al. (2026). *LLM Inference at the Edge: Mobile, NPU, and GPU Performance Efficiency Trade-offs Under Sustained Load.* arXiv:2603.23640. — Establishes real-world power and thermal constraints for on-device LLM operations; grounds the on-device training loop feasibility claim.

Public domain. No rights reserved. Build freely.

JONATHAN HORVAT · 2026